

Software Requirements Specification

FarmConnect Ghana — A Mobile-First Marketplace Connecting Ghanaian Smallholder Farmers to Buyers, Market Prices, and Agricultural Advisory Services

Conforms to the IEEE Std 830-1998 recommended practice for Software Requirements Specifications. Version 2.0 reconciles the original requirements baseline with the delivered, deployed system and records every approved scope deviation with its engineering rationale.

Course	CSCD602 — Advanced Software Engineering, Capstone Project
Institution	University of Ghana — Department of Computer Science
Group	Angry Nerds
Project title	FarmConnect Ghana
Document version	2.0 (post-implementation reconciliation)
Date	12 August 2026
Individual submission by	George Boadu Junior — 22427354

This individual submission is prepared by George Boadu Junior on behalf of Group Angry Nerds, per the CSCD602 individual-submission requirement. See "Group Information" overleaf for the full roster.

Group Information & Revision History

Group Information

#	Group member	Student ID	Major contribution
1	George Boadu Junior (this submission)	22427354	Requirements engineering & SRS authorship, system architecture & design, full-stack implementation (API + PWA), test authoring/execution, deployment engineering, and project documentation.
2	Marvin Kabutey	22425067	[Confirm with group before final submission]
3	Obed Borley Obuadey	22424177	[Confirm with group before final submission]
4	Richmond Gakpetor	22424720	[Confirm with group before final submission]
5	Isichei Phelim	22424142	[Confirm with group before final submission]
6	Edem Afflu	22424139	[Confirm with group before final submission]
7	Emmanuel Okyere Gyateng	22424179	[Confirm with group before final submission]
8	Edinam Ahadzie	22424539	[Confirm with group before final submission]
9	Desmond Techie	22424555	[Confirm with group before final submission]
10	Irene Hamilton	22424758	[Confirm with group before final submission]
11	Bernard Tetteh Djangbah	22424329	[Confirm with group before final submission]

NOTE ON INDIVIDUAL SUBMISSION

Every group member must submit this same core package individually through SAKAI LMS, editing the "major contribution" column and any first-person passages to reflect their own role and understanding, per the CSCD602 Individual Submission Requirement.

Revision History

Date	Version	Author(s)	Description
May 2026	1.0	Group Angry Nerds	Initial complete SRS for FarmConnect Ghana, drafted before implementation began.

Date	Version	Author(s)	Description
12 Aug 2026	2.0	George Boadu Junior (Group Angry Nerds)	Post-implementation reconciliation: added a Status column to every FR/NFR, corrected the architecture description to match the delivered stack, and added Section 10 documenting every approved deviation between v1.0 and the shipped system, with rationale.

Table of Contents

1	Introduction	
2	Overall Description	
3	Functional Requirements	
4	Non-Functional Requirements	
5	External Interface Requirements	
6	Other Requirements / Constraints	
7	System Architecture Overview	
8	Use Case Scenarios	
9	Traceability & Reconciliation with the Delivered System	
10	Appendices	

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for FarmConnect Ghana, a mobile-first web platform designed to connect smallholder farmers in Ghana directly with verified buyers and agricultural advisory services. It is the authoritative reference for the design, implementation, testing, and acceptance of the system, and is written to guide both the development team (Group Angry Nerds) and the course evaluators reviewing this capstone project. Version 2.0 additionally reconciles this baseline against the system that was actually designed, built, tested, and deployed (see Section 9).

1.2 Scope

FarmConnect Ghana is a system covering the following capabilities:

- User registration, authentication, and role-based profiles (Farmer, Buyer, Administrator; Extension Officer reserved — see 9.3).
- Creation, browsing, and management of produce listings, including photos, quantity, price, harvest date, and GPS location.
- Order placement, confirmation, and tracking between farmers and buyers, including a manual Mobile Money payment-reconciliation workflow (see 9.1).
- A live market price dashboard.
- An AI-powered advisory chatbot for crop guidance, weather context, and photo-based pest identification.
- SMS fallback channels for users on feature phones or with limited data access.
- Trust and rating mechanisms for completed transactions, and cooperative (co-op) group formation.
- An administrator portal for account verification, listing moderation, and payment-dispute resolution.

The system explicitly excludes: direct logistics/delivery dispatch management, crop insurance underwriting, and government subsidy administration, which remain candidates for future phases (see the companion *Maintenance & Future Evolution Plan*).

1.3 Intended Audience and Use

- **Development team** (Group Angry Nerds) — as the basis for system design and implementation.
- **Course instructors and evaluators** — to assess requirements completeness and engineering rigor.

- **Future maintainers** — as a reference for system behaviour and constraints.
- **Potential pilot partners** (e.g., agricultural extension officers, NGOs) — to understand system capabilities.

1.4 Product Scope and Business Context

Ghana has over 2.7 million smallholder farmers, many of whom lack reliable access to buyers, fair pricing information, and post-harvest support. Middlemen currently capture up to 60% of potential profit margins, and post-harvest losses reach 30–40% due to farmers' inability to sell produce before spoilage. FarmConnect Ghana addresses this gap with a low-cost, accessible digital marketplace tailored to the Ghanaian context — including support for low-end Android devices, SMS access, and local-language (Twi) support.

1.5 Definitions, Acronyms, and Abbreviations

Farmer	A registered seller of agricultural produce on the platform.
Buyer	A registered purchaser of produce, from individual consumers to wholesalers.
Listing	An individual produce post created by a farmer: crop type, quantity, price, and location.
MoMo	Mobile Money — a mobile-based payment method offered by telecom providers (MTN, Telecel, AirtelTigo).
OTP	One-Time Password, used for phone-based authentication.
SRS / FR / NFR	Software Requirements Specification / Functional Requirement / Non-Functional Requirement.
PWA	Progressive Web App — an installable, offline-capable web application.
JWT	JSON Web Token — signed token format used for session authentication.
Co-op	A cooperative group of farmers who can jointly attribute listings to the group.
Twi (Akan)	A widely spoken local language in Ghana, supported as an alternate UI language.

1.6 References

- IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*.
- MTN Mobile Money, Telecel Cash, and AirtelTigo Money — public product documentation (payment flows referenced for the manual-reconciliation design, §9.1).
- Google Gemini API documentation (`gemini-flash-latest`) — ai.google.dev.
- OpenWeatherMap Current Weather API documentation — openweathermap.org/api.
- Africa's Talking SMS API documentation (inbound webhooks) — africastalking.com.
- GiantSMS API documentation (outbound SMS, Ghana) — giantsms.com.

- FarmConnect Ghana project presentation, Group Angry Nerds, May 2026.

1.7 Document Overview

Section 2 describes product context, user classes, and operating environment. Section 3 specifies functional requirements. Section 4 specifies non-functional requirements. Section 5 describes external interface requirements. Section 6 lists constraints and assumptions. Section 7 summarises the architecture (detailed diagrams live in the companion *Design Documentation*). Section 8 lists use-case scenarios. Section 9 — new in v2.0 — reconciles every requirement against what was actually delivered. Section 10 holds appendices, including AI-assisted drafting disclosures.

2. Overall Description

2.1 Product Perspective

FarmConnect Ghana is a new, standalone system composed of a mobile-first Progressive Web App frontend, a backend REST API (Node.js/Express, TypeScript), and a data layer (PostgreSQL with Redis caching). It integrates with four external systems: an SMS gateway pair for outbound and inbound feature-phone messaging, an AI large-language-model provider for the advisory chatbot, and a weather data provider used to ground advisory answers. *It does not integrate with a Mobile Money payment gateway* — see §9.1 for the rationale and the manual-reconciliation design that replaces it.

2.2 Product Functions

- **Farmer Marketplace** — listing creation with photos, quantity, price, and location; buyer browsing and ordering.
- **SMS + App Access** — dual-channel access for smartphone and feature-phone users.
- **Live Market Prices** — a continuously updating commodity price feed.
- **Advisory Chatbot** — an AI-powered assistant for weather, pest, and crop-practice guidance, including photo-based pest identification.
- **Order Alerts** — in-app and SMS notifications for orders and price changes.
- **Co-op Groups** — farmer cooperatives that can jointly attribute listings to the group.
- **Ratings & Trust** — a 1–5 star mutual rating system feeding a per-user trust score.
- **Administration** — account verification/suspension, listing moderation, and payment-dispute resolution.

2.3 User Classes and Characteristics

User class	Characteristics
Farmer (primary)	Smallholder producer; may use a feature phone (SMS) or a low-end smartphone; varying literacy; primary language often Twi. Must link Mobile Money details before publishing listings.
Buyer (primary)	Individual consumer, retailer, or wholesaler; typically more comfortable with a smartphone; needs search, filtering, and trust signals before paying a stranger directly.
Administrator	Platform operator; verifies/suspends accounts, moderates listings, and arbitrates payment disputes. Seeded out-of-band (no self-service admin signup — see §9.3).

User class	Characteristics
Agricultural Extension Officer (reserved)	Named as a role in the original v1.0 baseline; the role constant exists in the data model but has no implemented routes, guards, or UI in the delivered system — a deferred, not a dropped, requirement (§9.3).

2.4 Operating Environment

- **Client:** Android/iOS smartphones (including low-RAM devices) via an installable Progressive Web App running in any modern mobile browser; basic feature phones via the SMS gateway.
- **Server:** A containerisable Node.js/Express application server, deployable to any Node-compatible host (see the *Deployment* section of the Project Documentation).
- **Database:** PostgreSQL 16 (primary data store) with Redis 7 (OTP storage, rate limiting, refresh-token allowlist, weather-cache).
- **Network:** Optimised for 3G connectivity typical of rural Ghana; offline-tolerant for previously-seen read data via Workbox service-worker caching.

2.5 Design and Implementation Constraints

- Must operate acceptably on 3G networks and devices with limited RAM/storage.
- No card or Mobile Money credentials may be stored on the platform — payment handling is a manual, human-verified hand-off (§9.1), not a stored-credential flow.
- UI must support English and Twi (Akan) at minimum, with a runtime language toggle.
- All critical farmer actions (listing produce, confirming payment) must have a working SMS fallback path with no smartphone required.

2.6 User Documentation

The system provides in-app onboarding screens, a bilingual UI, and an SMS `HELP` command for feature-phone users. The companion *User Manual* (submitted alongside this SRS) is the full end-user reference for every role.

2.7 Assumptions and Dependencies

- Assumes continued availability of the configured SMS gateway (GiantSMS outbound / Africa's Talking inbound), the Gemini API, and the OpenWeatherMap API — each degrades independently and predictably if unavailable (see §4.2, §9.2, §9.4).
- Assumes farmers and buyers possess a phone number capable of receiving SMS/OTP.
- Assumes buyers and farmers are willing to complete the Mobile Money leg of a transaction directly with each other, outside the app, and to act honestly when confirming/disputing a payment (the trust model underpinning §9.1).
- Assumes target pilot regions have at least intermittent 2G/3G mobile network coverage.

3. Functional Requirements

Each requirement is uniquely identified (FR-XX) for traceability to design, test cases, and the delivered code. The **Status** column (new in v2.0) records how the requirement was actually realised; every "Approved deviation" is explained in full in Section 9.

ID	Requirement	Description	Status
FR-01	Authentication & Registration	The system shall allow users to register via phone number with OTP verification, and to select a role (Farmer or Buyer) during onboarding. Returning users shall log in using their verified phone number and OTP, and shall remain signed in via a session token.	✓ As specified
FR-02	Produce Listings	Farmers shall create, edit, and remove produce listings, specifying crop type, quantity (kg), price per kg, harvest/availability date, one or more photos, and GPS location. Listings shall be marked Active, Pending, Sold, or Removed.	✓ As specified
FR-03	Search & Filtering	Buyers shall search and filter listings by crop type/category, free text, price range, and geographic radius.	⚠ Partial — voice-input search from v1.0 was not built (\$9.5)
FR-04	Order Placement & Confirmation	Buyers shall place an order against a listing. The system shall notify the farmer via in-app and SMS alert; the order proceeds through an explicit, auditable status lifecycle from placement to delivery confirmation.	✓ As specified
FR-05	Payments (Mobile Money)	The system shall support Mobile Money-based payment from buyer to farmer, with the platform never taking custody of funds, and a defined process for the farmer to confirm or reject a claimed payment.	📧 Approved deviation — manual reconciliation replaces gateway escrow (\$9.1)
FR-06	Live Market Prices	The system shall display commodity prices on a dedicated price dashboard, refreshed at a defined interval.	📧 Approved deviation — own simulated feed replaces a GCX API integration (\$9.2)

ID	Requirement	Description	Status
FR-07	AI Advisory Chatbot	The system shall provide an AI chatbot for crop advice, weather-aware guidance, and pest identification via photo upload.	 Approved deviation — Google Gemini replaces the originally-implied provider (§9.4)
FR-08	Ratings & Trust Scores	The system shall allow both farmers and buyers to rate each completed transaction on a 1–5 star scale, and shall aggregate ratings into a visible trust score per user.	 As specified (design detailed in §9.6)
FR-09	Notifications	The system shall send in-app and SMS notifications for order-lifecycle events and administrator actions.	 As specified
FR-10	Cooperative (Co-op) Groups	The system shall allow farmers to create or join cooperative groups, enabling listings to be attributed to the group.	 Implemented (scoped down from "joint negotiation" to attribution + leadership transfer — §9.6)
FR-11	SMS Fallback Channel	The system shall provide an SMS command interface allowing feature-phone users to register, link Mobile Money, list produce, check prices, view orders, and confirm payments without a smartphone app.	 As specified
FR-12	Multilingual Support	The system shall present its UI in English and Twi (Akan), with a visible language toggle and language selection during onboarding.	 As specified (farmer/buyer UI only — admin portal is English-only by design, §9.7)

ID	Requirement	Description	Status
FR-13	Administration & Moderation	Administrators shall verify/suspend accounts, moderate fraudulent listings, and resolve disputes raised between transacting parties.	 Implemented (concrete workflow designed in §9.3, since v1.0 specified only the intent)
FR-14 new	Payment Dispute Resolution	When a buyer disputes a rejected payment, the system shall route the dispute to an administrator, who shall resolve it by upholding the payment (order proceeds) or upholding the rejection (order cancels, listing reopens); both parties shall be notified of the outcome.	 Added in v2.0 — the concrete mechanism needed once FR-05's manual reconciliation was adopted (§9.1)

4. Non-Functional Requirements

4.1 Performance

ID	Requirement	Status
NFR-P1	Page load time shall not exceed 3 seconds on a typical 3G network connection.	✅ Met — Workbox precache + code-split PWA shell; see Testing Report §Performance
NFR-P2	The system shall support at least 10,000 concurrent users without service degradation.	⚠️ Not load-tested at this scale — capstone-scope limitation (§9.8), architecture is horizontally scalable
NFR-P3	API response time shall not exceed 500 ms for the 95th percentile of requests.	✅ Met in local/staging measurement — see Testing Report

4.2 Security

ID	Requirement	Status
NFR-S1	All client-server traffic shall be encrypted end-to-end using HTTPS/TLS.	✅ Enforced at the hosting layer (deployment guide, Project Documentation §Deployment)
NFR-S2	The system shall use JWT-based authentication, with short-lived access tokens and a revocable refresh mechanism.	✅ Met — 12h access / 30-day single-use, rotating refresh tokens via a Redis allowlist (Design Documentation §Auth Sequence)

ID	Requirement	Status
NFR-S3	No card or Mobile Money credentials shall be stored on FarmConnect servers.	✓ Met by construction — the platform never touches payment credentials (§9.1)

4.3 Usability

ID	Requirement	Status
NFR-U1	The application shall be usable on Android 6.0+ devices, including low-RAM/low-storage handsets.	✓ Met — plain PWA, no native runtime overhead
NFR-U2	All critical actions (listing produce, confirming payment) shall have a functioning SMS fallback path.	✓ Met — see FR-11
NFR-U3	The UI shall support English and Twi with accessible labels and a minimum 48dp tap target.	✓ Met — full i18n + accessibility pass (Phase 7, see Testing Report §Usability/Accessibility)

4.4 Reliability & Availability

ID	Requirement	Status
NFR-R1	The system shall maintain a 99.5% uptime SLA, excluding scheduled maintenance windows.	⚠ Aspirational at capstone scale — see Maintenance & Evolution Plan §Preventive Maintenance
NFR-R2	The system shall perform automatic daily backups, retained for a minimum of 30 days.	⚠ Depends on chosen host's managed-Postgres backup policy — documented per-provider in the Deployment Guide
NFR-R3	Core read-only features shall degrade gracefully and remain partially usable offline or under poor connectivity.	✓ Met — Workbox <code>NetworkFirst</code> / <code>CacheFirst</code> runtime caching + offline banner (Phase 7)

4.5 Maintainability & Portability

ID	Requirement	Status
NFR-M1	The backend shall be organised into clearly separated modules (Auth, Listings, Orders, Prices, Advisory, Notifications, Co-ops, Admin, SMS Gateway) to simplify maintenance and future extraction into independent services.	☒ Met — module-per-domain layout, see Design Documentation §Component View
NFR-M2	The frontend shall run on modern mobile browsers without requiring app-store installation.	☒ Met — installable PWA

5. External Interface Requirements

5.1 User Interfaces

- Mobile-first responsive Progressive Web App covering Onboarding/Login/OTP, Role Selection, Farmer Home/Listings/Momo-Setup, Buyer Marketplace/Checkout, Orders & Payment Submission, Advisory Chat, Prices, Notifications, Profile, and a separate Admin Portal (Users/Listings/Disputes).
- SMS text-command interface for feature-phone users: `HELP` , `REG` , `MOMO` , `LIST` , `PRICE` , `ORDERS` , `CONFIRM` .

5.2 Hardware Interfaces

- Device camera — for produce photo capture and pest-identification photo uploads.
- Device GPS/location services — for listing geolocation, buyer distance-based search, and weather-context lookups.

5.3 Software Interfaces

Interface	Purpose
Google Gemini API (<code>gemini-flash-latest</code>)	Text advisory answers and vision-based pest/disease photo identification.
OpenWeatherMap Current Weather API	Supplies weather context folded into advisory chatbot answers (Redis-cached ~45 min).
GiantSMS API	Outbound SMS — OTP codes, order/admin notifications, SMS-command replies.
Africa's Talking inbound webhook	Delivers inbound feature-phone SMS commands to <code>POST /api/sms/inbound</code> .

DEVIATION FROM V1.0

v1.0 named a single "MTN MoMo / AirtelTigo Money API" and a single "GCX API" as software interfaces for automated payment and pricing. Neither is integrated in the delivered system — see §9.1 and §9.2.

5.4 Communication Interfaces

- HTTPS REST API (JSON) for all client-server requests.
- SMS protocol via the two telecom gateway partners above, for feature-phone communication.

DEVIATION FROM V1.0

v1.0 specified WebSocket connections for real-time order/price updates. The delivered system uses REST polling (React Query, 30s stale-time) plus an in-app notification inbox instead — sufficient at this scale and considerably simpler to operate reliably on constrained mobile networks; a WebSocket/push upgrade is listed as a future enhancement (Maintenance & Evolution Plan §Scalability).

6. Other Requirements / Constraints

6.1 Regulatory Constraints

- The manual Mobile Money hand-off is designed to avoid, not comply with, Bank of Ghana e-money issuer/PSP licensing requirements — FarmConnect never takes custody of funds, so it does not act as a payment service provider (this was the deciding factor behind the §9.1 deviation, not merely an engineering convenience).
- User data handling is designed with Ghana's Data Protection Act, 2012 (Act 843) in mind: phone numbers and Mobile Money details are collected only for their stated purpose, and there is no card-data storage of any kind.

6.2 Project Constraints

- Delivered as a capstone project within a single academic semester; scope is bounded by Section 3.
- Engineering judgement (documented in Section 9) was exercised wherever v1.0 specified an integration this team could not obtain production access to within the project timeline (a real MoMo merchant API, a real GCX feed) or where the SRS was silent on implementation detail (co-ops, admin, ratings).

6.3 Assumptions

- Target pilot region(s) have at least intermittent mobile network coverage sufficient for SMS and occasional 3G data.
- Farmers and buyers possess a phone number capable of receiving OTP and SMS messages.
- Buyers and farmers will act in good faith on the manual Mobile Money hand-off, with the dispute-resolution workflow (FR-14) as the backstop when they don't.

7. System Architecture Overview

The system follows a three-tier architecture. Full architecture, component, and deployment diagrams are provided in the companion *Design Documentation* — this section summarises the tiers for requirements traceability.

Layer	Components
Frontend	Vite + React + TypeScript Progressive Web App — Farmer UI, Buyer UI, Admin Portal, bilingual (EN/Twi), offline-caching via Workbox.
Backend	Node.js / Express (TypeScript) REST API — Auth, Users, Listings, Orders, Prices, Advisory, Notifications, Co-ops, Admin, SMS-Gateway, Uploads modules.
Data / External	PostgreSQL 16 (primary store, via Prisma ORM), Redis 7 (cache/session/rate-limit), Google Gemini (AI advisory), OpenWeatherMap (weather context), GiantSMS (outbound SMS), Africa's Talking (inbound SMS webhook).

DEVIATION FROM V1.0

v1.0 named the frontend "React Native" and implied a native mobile build. The delivered frontend is a plain React web PWA (Vite + TypeScript) — installable and offline-capable without native tooling or app-store distribution, which better fits a project of this scope and the low-end-Android/no-app-store-friction target audience. See §9.7.

8. Use Case Scenarios

Full actor/use-case diagrams and step-by-step flows are provided in the Design Documentation. Summarised here for requirements traceability:

Use case	Actor(s)	Description
UC-01 List Produce	Farmer	Farmer completes Mobile Money onboarding, then creates a listing with crop details, photos, and location; publishes it to the marketplace.
UC-02 Search & Order	Buyer	Buyer searches/filters listings, views a farmer's profile and trust score, and places an order.
UC-03 Pay & Confirm	Buyer, Farmer	Buyer pays the farmer directly via Mobile Money and submits a transaction ID; farmer confirms or rejects it against their own Mobile Money history; buyer confirms delivery on receipt.
UC-04 Get Crop Advice	Farmer, Buyer	User opens the advisory chatbot, optionally uploads a photo of a crop/pest issue, and receives a weather-aware answer.
UC-05 Form a Co-op	Farmer	Farmers create or join a cooperative via a short join code to pool listings under a shared identity.
UC-06 Resolve a Dispute new	Buyer, Farmer, Admin	Buyer disputes a rejected payment; an administrator reviews evidence and resolves it, notifying both parties.
UC-07 Moderate the Platform new	Admin	Administrator verifies/suspends accounts and force-moderates listings reported as fraudulent.

9. Traceability & Reconciliation with the Delivered System

This section is the single most important addition in v2.0. Every deviation between the original v1.0 requirements baseline and the delivered system is recorded here with its rationale, so an examiner can trace requirements engineering decisions all the way to shipped code — not just read two documents that quietly disagree.

9.1 FR-05 — Payments: manual Mobile Money reconciliation, not gateway escrow

v1.0 specified in-app escrow via the MTN MoMo / AirtelTigo Money merchant APIs, with funds released to the farmer only on buyer delivery confirmation. Obtaining production merchant/PSP API access within a single-semester capstone timeline was not realistic, and doing so would have made FarmConnect itself a regulated payment intermediary under Bank of Ghana rules. The delivered design instead:

1. Requires a farmer to link their Mobile Money network, phone number, and registered account name before publishing any listing.
2. Shows those details to a buyer at checkout; the buyer pays directly via their own MoMo app/USSD, *outside* FarmConnect, then submits the transaction ID they received.
3. Lets the farmer confirm or reject that transaction ID against their own Mobile Money SMS/history.
4. Adds FR-14 (§3) as the dispute backstop for a buyer who insists they paid and a farmer who disputes it, arbitrated by an administrator.

FarmConnect never touches, stores, or moves money at any point — it is a matching and reconciliation layer, which is both a safer regulatory posture and an honest reflection of what could be built and demonstrated in this project's timeframe.

9.2 FR-06 — Prices: a self-seeded simulated feed, not a GCX integration

No public, developer-accessible Ghana Commodity Exchange price API exists to integrate with at the time of writing. The delivered `PriceFeed` service instead seeds realistic per-crop prices on boot and nudges each by up to $\pm 3\%$ every 5 minutes, so the price dashboard, price-change indicators, and SMS `PRICE` command all behave exactly as a real feed would from the UI's point of view. The service is isolated behind one module boundary (§9.8/NFR-M1), so swapping in a real GCX (or any other) feed later is a contained change, not a rewrite.

9.3 FR-13 — Administration: a concrete design for an SRS sentence

v1.0 gave FR-08 (ratings), FR-10 (co-ops), and FR-13 (admin) one sentence of intent each, with no data model, workflow, or screen spec. Rather than leave them unspecified, the team designed and built a concrete admin portal (`/admin/users` , `/admin/listings` , `/admin/disputes`) gated by a real `ADMIN` role. There is deliberately no self-service admin signup — the public role-selection endpoint only ever issues `FARMER` / `BUYER` , and admin accounts are seeded out-of-band by whoever controls the deployment (a one-line CLI command against the production database), then log in through the exact same phone+OTP flow as everyone else. The `EXTENSION` (Agricultural Extension Officer) role named in v1.0's user classes exists as a reserved enum value in the data model but has no routes, guards, or screens — a deliberately deferred, not abandoned, requirement (see the Maintenance & Future Evolution Plan).

9.4 FR-07 — Advisory chatbot: Google Gemini, not an unspecified/Claude-style provider

v1.0 left the AI provider unspecified beyond "AI-powered." The delivered system uses Google Gemini (`gemini-flash-latest`) specifically because it has a genuinely free developer-side tier with no credit card and no per-end-user account requirement — the right fit for a chatbot shared by many farmers/buyers, none of whom should need their own AI provider account. Weather context (OpenWeatherMap, Redis-cached) is folded into the system prompt when the caller's location is available; if the weather lookup or the Gemini call itself is unconfigured, the advisor either degrades gracefully (weather) or fails loudly with a clear error (no Gemini key) rather than fabricating a response.

9.5 FR-03 — Search: text/filter search shipped; voice input did not

v1.0 mentioned voice-input search as a stretch capability. Text search, category/crop filters, price-range filters, and geo-radius filtering are all implemented; voice input was descoped for time and is listed as a future enhancement in the Maintenance & Future Evolution Plan.

9.6 FR-08 / FR-10 — Ratings and co-ops: the concrete workflow rules

Ratings: a rating can only be left on an order once it reaches `delivered` , one rating per (order, rater); each new rating recomputes the rated user's trust score and rating count in the same database transaction as the insert, so the two figures can never drift apart. Co-ops: a farmer belongs to at most one co-op at a time; a listing can optionally be attributed to the farmer's co-op instead of inventing a separate bulk-order/negotiation subsystem the SRS never specified; leaving a co-op promotes the earliest-joined remaining member to leader, or dissolves the co-op if the leader was the sole member.

9.7 Platform & interface deviations

The frontend is a plain React web PWA rather than a native/React Native build (§7), and real-time updates use REST polling rather than WebSockets (§5.4) — both are load-bearing simplifications,

not cosmetic ones, and both are revisited as future work in the Maintenance & Future Evolution Plan. The admin portal is English-only by deliberate exception to FR-12: it is an internal staff tool, not part of the farmer/buyer-facing product the bilingual requirement targets.

9.8 Requirements not fully verified at capstone scale

NFR-P2 (10,000 concurrent users) and NFR-R1/R2 (99.5% uptime SLA, daily backups) describe production-operations targets that this capstone submission has not load-tested or operated long enough to demonstrate empirically. The architecture does not preclude meeting them (stateless API instances behind a load balancer, a managed Postgres with point-in-time recovery), but that verification is explicitly out of scope for a single-semester deliverable and is called out here rather than silently claimed. See the Testing Report and the Maintenance & Future Evolution Plan for what was actually measured and what is recommended before any real pilot.

10. Appendices

Appendix A — AI Prompts Used in SRS Drafting

In accordance with course AI-disclosure requirements, the following prompts (submitted via Claude, claude.ai) were used by George Boadu Junior to assist in drafting the v1.0 SRS, which the team then reviewed and finalised, and which this v2.0 revision reconciles against the delivered system:

1. "Act as a senior software engineer. Write a full Software Requirements Specification (SRS) for a mobile-first web app called FarmConnect Ghana that connects smallholder Ghanaian farmers directly to buyers. Include: Introduction, Scope, Definitions, Functional Requirements (with FR IDs), Non-Functional Requirements, and Constraints. Use IEEE 830 format."
2. "Update the SRS to include Ghana-specific context: mobile money payment integration (MTN MoMo, AirtelTigo), SMS fallback for feature phones, Twi language support, and Ghana Commodity Exchange (GCX) price feed integration."
3. "Expand the Non-Functional Requirements section. Add measurable targets for Performance (response time, concurrent users), Security (encryption, authentication), and Reliability (uptime SLA, offline support)."

Appendix B — Prototype Design Prompts

The following prompts were submitted via Figma Make to produce the early UX/flow/copy prototype referenced during design (screen names, i18n keys, and the colour palette were carried forward from it into the real implementation; it was not pixel-matched):

1. "Create a mobile app UI for FarmConnect Ghana — a farmer-to-buyer agricultural marketplace. Style: clean, modern, green colour palette. Screens needed: Onboarding/Login, Farmer Dashboard, Create Listing, Buyer Marketplace, AI Advisory Chatbot. Target users: Ghanaian smallholder farmers with low-end Android phones."
2. "Redesign the Farmer Dashboard screen. Add: a personalised greeting at the top, a 3-stat summary row (Active Listings, Pending Orders, Earnings), produce listing cards with status badges (Active / Pending / Sold), and a bottom navigation bar with icons for Home, Listings, Orders, Prices, and Profile."
3. "Create the Buyer Marketplace screen. Include: a search bar with voice input icon, horizontal filter chip row (All / Grains / Vegetables / Fruits), and listing cards showing: crop emoji, crop name, farmer name and region, quantity, price per kg, distance, and a verified farmer badge. Ensure large tap targets throughout."
4. "Update all screens to support Twi (Akan) as an alternative language. Add a language toggle button top-right. Increase minimum button height to 48dp. Add high-contrast mode. Ensure all interactive elements have accessible labels."

Appendix C — Group "Angry Nerds"

Full roster and per-member contributions are recorded in "Group Information" at the front of this document. This individual submission package was assembled and authored by George Boadu Junior; each other group member submits the same core deliverables individually, annotated with their own contribution and understanding of the system as required by the CSCD602 Individual Submission Requirement.

University of Ghana, Department of Computer Science — Advanced Software Engineering (CSCD602), Capstone Project.